PIFA: Adversarially Trained, Forgery-Resistant Attribution of Virtualization-Obfuscated APT Malware

Xubo Fang[1,†,*], Haochen Dong[2,†], Lei Zhong[3,†], Jiaqing Wang[2,*], and Sicheng Han[4]

Abstract—Attributing malware to the advanced persistent threat group that produced it guides the response to an intrusion. This attribution must stay reliable after commercial virtualization obfuscation has rewritten the code of a file. It must also resist an adversary who forges the metadata of the file. However, existing static attributors read byte or header features that such an adversary can rewrite. Their evaluations also shared executables or builds between training and testing. Therefore, we propose the Perturbation-Invariant Forgery-Resistant Attribution model, which reads every field of the portable executable header with gradient-boosted trees. The model trains these trees on forged, false-flag, and edited copies of each executable, all labeled with the true group. A split on a manipulated field then separates samples of the same group, which drives the trees to use such a field only where it agrees with the unchanged fields. We also develop a multi-view Transformer that learns from each executable paired with its protected version. We evaluated both methods on executables of nine groups and on their versions protected with VMProtect, with folds grouped by build. On unseen builds, the proposed model attributed protected executables with a macro-averaged F1 score of 91.2%. Its score stayed at 90.9% or higher under forged, false-flag, and edited headers. Without the copies, the score of the trees fell from 92.5% to 48.3% under forgery. These results show that training on manipulated headers yields an attribution of protected malware that survives header forgery.

Index Terms—Adversarial training, code obfuscation, gradient boosting, malware attribution, portable executable, robustness, virtualization obfuscation.

1 Introduction

Machine learning is widely studied for malware detection and classification [1]. Static classifiers decide from the bytes of an executable without running it. They answer two questions, namely whether a file is malicious and which threat group produced it. Attributing Advanced Persistent Threat (APT) malware to its group addresses the second question. The attributed group guides how defenders respond to an intrusion [2]–[4]. Commercial virtualization obfuscators such as VMProtect and Themida [5] make this attribution harder. They translate selected code into bytecode for a virtual machine that is embedded in the protected file [6]. The translation rewrites the byte statistics that a static classifier learns from. In the APT executables of our corpus, VMProtect raised the median file size from 180 KiB to 9,648 KiB. It also raised the mean entropy of 256-byte chunks from 4.78 to 6.93 bits per byte. An adversary can further rewrite the descriptive metadata of a protected file or copy it from another group. A classifier that relies on such metadata then misattributes the sample.

Previous studies have rendered malware as images and classified the images with texture features [7], Convolutional Neural Networks (CNNs) [8], attention modules [9], and Vision Transformers [10], [11]. For packed executables, Aghakhani *et al.* [12] showed that static classifiers did not generalize to packers absent from their training data. In our previous work [13], we protected APT malware with VMProtect and classified images of the protected files with Mal-CNN, a depth-wise separable CNN with spatial attention. That work reported an accuracy of 93.55%. However, it left three problems open. First, its released image splits place augmented images of 820 of the 1,015 test executables in the training set. Many executables of the corpus also share one build. A classifier can therefore recognize an individual executable

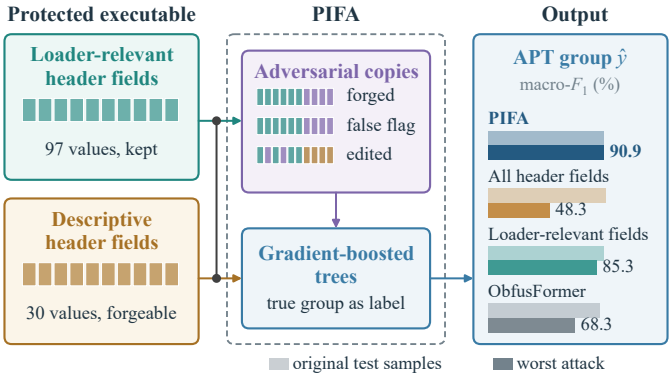


Fig. 1. Concept of the Perturbation-Invariant Forgery-Resistant Attribution (PIFA) model. PIFA reads the loader-relevant and descriptive fields of the Portable Executable header of a protected executable. It trains gradient-boosted trees on adversarial copies with forged, false-flag, and edited fields, all labeled with the true Advanced Persistent Threat (APT) group. Purple cells mark changed fields. Light bars give the macro-averaged F_1 score (macro-F_1) on the original protected test executables of the folds grouped by build, and dark bars give the worst case over the three manipulations.

[1]*Xubo Fang is with the Graduate School of Natural Science and Technology, Kanazawa University, Kanazawa 920-1192, Japan (e-mail: boukuhaku1117@gmail.com).*
[2]*Haochen Dong and Jiaqing Wang are with the School of Oceanography, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: jack.hc.dong@gmail.com; jiaqing.wang@sjtu.edu.cn).*
[3]*Lei Zhong is with the College of Mechanical and Electrical Engineering, Hohai University, Changzhou 213022, China (e-mail: cgqzl520@163.com).*
[4]*Sicheng Han is with the Department of Computer Software, Hanyang University, Seoul 04763, South Korea (e-mail: 244612453sone@gmail.com).*
†*Xubo Fang, Haochen Dong, and Lei Zhong contributed equally to this work.*
**Corresponding authors: Xubo Fang; Jiaqing Wang.*

instead of attributing its group. Second, it remained unknown which parts of a protected file still identify its group. It also remained unknown which of these parts an adversary can rewrite, for example by forging the time stamp or by copying the metadata of another group. Third, each model learned from one protection state. A model trained on plain executables attributes protected executables poorly.

To our knowledge, no published attributor of protected malware resists forged header fields. Published evaluations of such attributors also do not keep the executables of one build in one fold. Motivated by this, we propose the Perturbation-Invariant Forgery-Resistant Attribution (PIFA) model for the static attribution of protected APT malware. PIFA divides the Portable Executable (PE) header into loader-relevant fields, which Windows uses to load and run a program, and descriptive fields, which the loader ignores. VMProtect keeps most loader-relevant fields. An adversary cannot copy them from another group without changing how the program runs. The descriptive fields also carry a group signal, but an adversary can forge them or copy them from another group. PIFA therefore reads every header field with gradient-boosted trees. It trains these trees on forged, false-flag, and edited copies of every executable, all labeled with the true group (see Fig. 1). Besides, we develop the Obfuscation-Paired Multi-View Transformer (ObfusFormer) to test whether a learned representation of the executable can match PIFA. ObfusFormer trains on each executable together with its protected version, which it treats as a second view of one sample in the manner of contrastive learning [14]. To measure both methods without leakage, we rebuild our earlier corpus with folds that keep every build together, with a split by time, and with test samples whose headers are forged, copied from another group, or edited. We validate detection on 46,245 images derived from the EMBER benchmark [15] and attribution on 2,352 APT executables of nine groups, 2,187 of which have a protected version.

The main contributions are summarized below.

- We propose PIFA, an attributor that trains gradient-boosted trees on adversarial copies of every executable with forged, false-flag, and edited header fields. On unseen builds of nine APT groups, PIFA attributed VMProtect-protected executables with a macro-averaged F_1 score (macro-F_1) of 91.2%. Under the three manipulations, its macro-F_1 stayed at 90.9% or higher, the highest worst case of the compared methods.
- We develop ObfusFormer, a multi-view Transformer with view-specific tokenizers and an obfuscation-paired contrastive loss. It can read the loader-relevant fields, the byte distribution, and the two ends of the file, and its most accurate configuration reads the loader-relevant fields alone. This configuration attributed the protected executables with a macro-F_1 of 87.5%, which is 23.4 points above the most accurate baseline network.
- We rebuild our earlier corpus of VMProtect-protected APT malware into folds grouped by build, a split by time, and three sets of manipulated test samples. Splitting the augmented images image by image raised the accuracy of Mal-CNN on protected malware from 16.5% to 81.2%. The released split therefore rewards the recognition of individual executables.

Our project page will link to the code and the derived datasets upon publication.[1]

2 Related Works and Existing Gaps

2.1 Image and Byte Representations of Executables

Static learning-based malware analysis maps the bytes of an executable to a label without running it. Nataraj *et al.* [7] rendered malware binaries as gray-scale images and classified them with texture features, reporting an accuracy of 98% on 9,458 samples from 25 families. Kalash *et al.* [8] replaced the texture features with a deep CNN. Yakura *et al.* [9] introduced attention to this representation and extracted byte regions that characterize a family from the attention map. Vision Transformers for malware images later added self-supervised pretraining [10] and cross-scale embedding with adaptive token masking [11]. In parallel, Raff *et al.* [16] trained a model on the raw byte sequences of whole executables, which they named MalConv. A third line of work summarizes a whole file by its byte distribution. Saxe and Berlin [17] computed a two-dimensional histogram of byte values and local entropy and fed it, with other static features, to a deep neural network. Anderson and Roth [15] released EMBER, a benchmark of static features extracted from 1.1 million Windows executables. Its reference classifier is a gradient-boosted decision tree model [18]. A fourth line of work reads the PE header. Raff *et al.* [19] trained neural networks on the raw bytes of part of the PE header and detected malware more accurately than a model on explicitly parsed header features. Webster *et al.* [20] extracted the Rich header from 964,816 malware samples and used it to identify similar malware and builds made in different build environments.

These representations share three limitations. First, the image-based studies resize a whole file into one fixed network input. Their Transformers then split that image into square patches that join bytes one image width apart in the file. Second, the byte-sequence models read a protected executable of several megabytes as one long sequence, in which the code added by the protector dominates. The distribution features discard the byte order entirely. Third, the header studies do not separate the fields that Windows uses to load a program from the descriptive fields that an adversary can rewrite. None of these studies measured which representation still identifies the group of an executable after virtualization.

2.2 Malware Analysis Under Virtualization Obfuscation

Virtualization obfuscation translates selected program code into bytecode for a virtual machine that is generated for, and embedded in, the protected program [6]. Sharif *et al.* [21] presented Rotalume, which recovered the bytecode syntax and semantics of malware emulated with VMProtect and Code Virtualizer from x86 execution traces. Xu *et al.* [22] simplified partially virtualized binaries with VMHunt. Li *et al.* [5] extracted 1,915 verified instruction-mapping rules from four commercial obfuscators with chosen-instruction attacks. On the learning side, Aghakhani *et al.* [12] packed 50,724 executables with nine packers, none of which was VMProtect. They reported false-positive rates of up to 83.06% when tElock, PECompact, or kkrunchy was withheld from training. Gibert *et al.* [23] investigated how packing affects static machine-learning detectors, particularly visualization-based ones, and concluded that packing limits current static systems. Han *et al.* [13] protected the executables of ten APT groups with

1. https://panzerjagerwang.github.io/pifa-apt/.

VMProtect and classified images of the protected files with MalCNN. For Android applications, Wu *et al.* [24] used contrastive learning to reduce the difference that obfuscation introduces. Their IFDroid classified images of function call graphs and kept an F_1 score of 98.4% on 69,421 obfuscated samples.

Deobfuscation recovers program semantics, but each sample requires execution traces or symbolic analysis [21], [22]. Li *et al.* [5] also found three state-of-the-art deobfuscation techniques insufficient against modern commercial obfuscators. The learning studies on Windows executables treat protected samples either as a separate dataset [13] or as a held-out condition [12]. IFDroid [24] relies on function call graphs recovered by program analysis, whereas virtualization replaces the protected functions with bytecode for an embedded interpreter [6]. None of these studies paired a Windows executable with its virtualization-protected version during training.

2.3 Evaluation Protocols for Malware Learning

Evaluation protocols decide whether a reported accuracy reflects performance on unseen malware. Arp *et al.* [25] studied 30 papers from top security venues and showed how pitfalls in design and evaluation lead to unrealistic performance. Pendlebury *et al.* [26] showed that spatial and temporal bias inflated the results of Android malware classification. Kaufman *et al.* [27] defined leakage as the use of information about the target that should not be legitimately available during learning. Boot *et al.* [3] released 3,594 samples from 12 APT groups and compared random forests with fully connected networks on sandbox and VirusTotal reports. Gray *et al.* [4] surveyed malware authorship attribution and noted the lack of assurance about how the APT dataset of Boot *et al.* was labeled. Laurenza *et al.* [28] triaged APT malware with one-class classifiers on static features. Haddadpajouh *et al.* [29] attributed payloads of five APT families with a multi-view fuzzy consensus model and found multi-view attribution more accurate than single-view attribution.

Image datasets of APT and protected malware contain a few hundred executables per group. Han *et al.* [13] enlarged theirs with duplication and geometric augmentation. If the augmented images are divided into training and test sets image by image, images derived from one executable can fall into both sets, which is a form of the leakage defined in [27]. Detection studies on EMBER-derived data face a different pitfall. The EMBER training set contains unlabeled samples [15] that must not be merged into the benign class. On EMBER-derived data, Han *et al.* [13] also compared networks with one another rather than with the gradient-boosted reference model of the benchmark. Malware collections further contain many executables of one build, which share their header fields. A random split can place such executables in both the training and the test set. Adversaries can modify a PE file without changing its behavior, for example by editing the DOS header or by injecting content into new sections [30], [31]. Header fields that a classifier relies on can therefore be forged. Several defenses of malware detectors against such evasion have been proposed. Li *et al.* [32] proposed Principled Adversarial Malware Detection (PAD), which adversarially trains a detector together with a convex network that flags adversarial inputs. Mimicry attacks guided by benign applications still lowered the accuracy of PAD on Drebin to 74.27% or less. Zhou *et al.* [33] placed a denoising autoencoder before an Android malware detector to purify perturbed feature vectors and reported an accuracy of at least 90.91% against 37 evasion attacks. Rashid and Such [34] found that most moving target defenses were no more robust than one adversarially trained network. In their study, a query attack reached an evasion rate of 94% on Windows malware described by its application programming interface calls. Abusnaina *et al.* [35] injected benign control-flow subgraphs into Internet of Things malware without changing its behavior and made a CNN classifier label all of it as benign. These defenses and attacks concern detectors that decide whether a sample is malicious. None of these studies trained an attributor on executables with manipulated header fields.

3 Problem Formulation and Preliminaries

3.1 Attribution of Unseen Builds Under Header Manipulation

An executable is a byte string $x = (b_1, \ldots, b_L)$ with $b_l \in \{0, \ldots, 255\}$, where L denotes the file length in bytes. A virtualization protector is a mapping $O(\cdot)$ that returns an executable $O(x)$, which behaves like x but in which selected code of x is replaced with bytecode for an embedded interpreter. We study two static tasks in which a classifier observes only the bytes of a sample and never executes, unpacks, or disassembles it. In detection, the classifier predicts whether an unseen sample is benign or malicious, $y^{\mathrm{d}} \in \{0, 1\}$. In attribution, the classifier receives x or $O(x)$ without its protection state and predicts the APT group $y \in \{1, \ldots, K\}$ that produced it, where K denotes the number of groups.

For attribution, the training data are

$$\mathcal{D} = \{(x_i, O(x_i), y_i)\}_{i \in \mathcal{P}} \cup \{(x_i, y_i)\}_{i \in \mathcal{U}}, \tag{1}$$

where $\mathcal{D}$ denotes the training set, y_i denotes the group of executable i, $\mathcal{P}$ denotes the indices of executables that have a protected version, and $\mathcal{U}$ denotes the indices of executables available only in plain form. Executables compiled from one build share their header fields, so a test executable whose build also occurs in training can be recognized rather than attributed. We therefore assign each plain executable x a build key $\kappa(x)$, the tuple of the time stamp, the code size, the size of initialized data, the entry-point address, and the image size in its PE header. We partition the executables into five folds stratified by group, with all executables of one build key in one fold. For fold q, the test set contains x_i for every index i in the fold and $O(x_i)$ for every such $i \in \mathcal{P}$. Neither version of a test executable, nor any executable of the same build, enters training or model selection. To test the attribution of builds made after the training builds, we also split by time. For each group, the latest builds by compile time stamp that hold about 20% of its executables form the test set. We compare three training protocols on the same folds. The Plain protocol trains on x_i only, the VM protocol on $O(x_i)$ only, and the Joint protocol on both. We test every model on plain and on protected test samples.

Our threat model has a defender who trains the classifier and owns a copy of the protector, so that every training executable can be protected. The adversary protects a malware sample with the same protector. After protection, the adversary may rewrite every header field that does not affect how Windows loads and runs a user-mode executable, such as the time stamp, the linker version, the section names, and the Rich header. The adversary may also copy these fields from malware of another group, as copied Rich headers have been used to point attribution at

another group. Among the loader-relevant fields, the adversary may edit those whose values do not change how the program runs, such as informational flags and the stack and heap sizes. Such functionality-preserving edits of PE files are known from attacks on static malware classifiers [31], [36]. Changes that rebuild the section layout or move the entry point lie outside this model. The adversary knows neither the training data nor the classifier.

For attribution, we report accuracy and the macro-averaged F_1 score

$$F_1^{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \frac{2P_k R_k}{P_k + R_k}, \tag{2}$$

where F_1^{macro} denotes the macro-averaged score, and P_k and R_k denote the precision and recall of group k, respectively. We compute it on the original test samples and on the same samples after each of the three manipulations of Section 4.2. For detection, we report accuracy, the F_1 score, the Area Under the Receiver Operating Characteristic Curve (ROC-AUC), and the True-Positive Rate (TPR) at a False-Positive Rate (FPR) of 1%.

Protection changes the bytes that image and byte classifiers read, as shown in Fig. 2(a). Of the 2,574 protected files, 94.1% had a size from 8 to 12 MiB, and 97.6% had a mean chunk entropy from 6.5 to 7.2 bits per byte. A protected executable of median size resized to a 128×128 image therefore averages about 603 bytes per pixel. The PE header, in contrast, keeps most of its fields, as shown in Fig. 2(b). In all 2,574 plain-protected pairs, the protected file had the time stamp, the linker version, the image base, the section and file alignments, the subsystem, the Dynamic-Link Library (DLL) characteristics, and the code size of its source. VMProtect changed the entry point, the image size, and the number of sections of every file, to which it added a median of three sections. The section table of 1,966 protected files, or 76.4%, still listed every section name of the source. VMProtect removed the Rich header, which 2,387 of the 2,747 plain executables contained. The header of a protected file therefore keeps most fields of its source, among them fields that an adversary can rewrite.

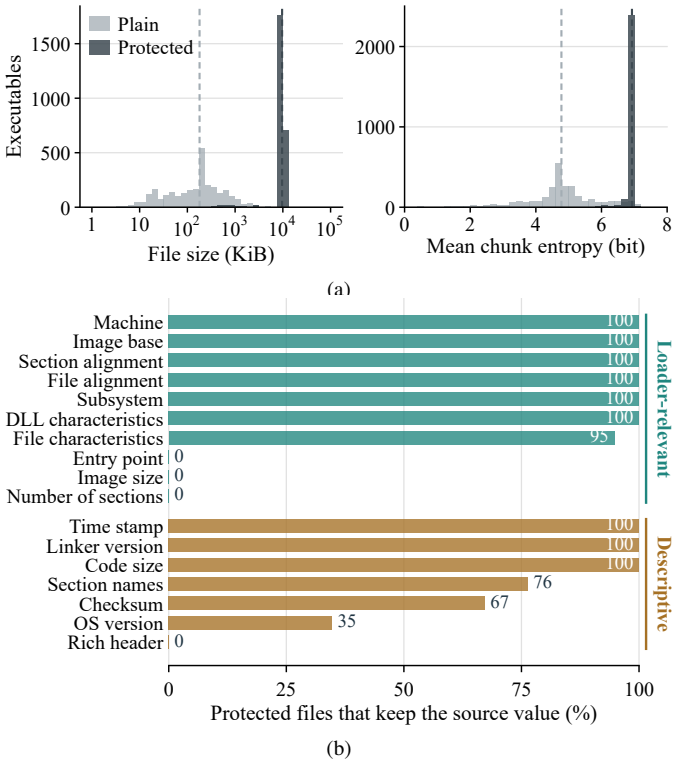


Fig. 2. Effect of VMProtect on the APT executables. (a) File size and mean chunk entropy of the 2,747 plain and 2,574 protected executables. Dashed lines mark the median size and the mean entropy. (b) Share of the 2,574 plain-protected pairs in which the protected file keeps the value of each header field of its source. For the Rich header, the share is over the 2,282 pairs whose source has one.

3.2 Notation, Header Fields, and Compared Methods

EMBER describes each executable by a vector of static features [15], which has 2,381 entries of 32-bit floating point in feature version 2 of the EMBER code. The detection images in the archive released with [13] come from a renderer that reads the 9,524 bytes of this vector, repeats each byte six times, truncates the sequence at 224×224 pixels, and rescales the result to $[0, 255]$. Each image therefore contains the first 8,363 bytes of the feature vector. We invert the repetition exactly and fold these bytes, padded with zeros, into a native image of 96×96 pixels for the image networks. We also decode them into the first 2,090 feature values, which run from the byte histogram to 1,147 of the 1,280 hashed import features. The decoded byte histogram sums to one, as in EMBER, for 93.1% of the training images and 95.3% of the test images. In the other images, the rescaling of the renderer altered the bytes.

For attribution, we parse the PE header of every executable from its first 64 KiB. We split its fields into two sets by whether they affect how Windows loads and runs a user-mode executable. The 97 loader-relevant values $\mathbf{r}(x)$ describe the machine, the number of sections, the PE format, and the file characteristics. They also hold the sizes of the optional header, of the image, and of the headers, together with the entry point, the image base, and the section and file alignments. Further loader-relevant values give the subsystem and its version, the DLL characteristics, the stack and heap sizes, and the data directories other than the certificate and debug entries. The last of them give the virtual size, virtual address, raw size, raw offset, and characteristics of the first ten sections. The 30 descriptive values $\mathbf{c}(x)$, which the loader ignores, hold the time stamp, the pointer and count of the symbol table, and the linker, operating-system, and image versions. They also hold the code and data sizes and bases, the certificate and debug directory entries, the names of the first ten sections, and the presence of a Rich header. The Rich header is a block of build-environment metadata between the DOS stub and the PE header [20]. The descriptive values further include the checksum, which Windows validates only for drivers and for DLLs loaded at boot time or into critical processes. The image networks read a global statistics view $\mathbf{G}$, which divides the whole file into $M = 256 \times 256$ cells of consecutive bytes. Its three channels hold, for each cell, the mean byte value, the Shannon entropy of the 256-byte chunk at the cell scaled by 255/8, and the fraction of printable ASCII bytes scaled by 255. Table 1 defines the remaining symbols of the paper.

We compare PIFA with gradient-boosted trees that read, without adversarial copies, all header fields, the loader-relevant fields alone, the descriptive fields alone, or the byte-distribution view $\mathbf{d}$ of Section 4.3. Eight baseline networks, ObfusFormer, and a nearest-neighbor rule complete the comparison. We reimplemented the baseline CNN and Mal-CNN of Han *et al.* [13] from the layer table of their paper. The CNN stacks four 3×3 convolution blocks with batch normalization and max pooling before a fully connected layer of 1,024 units. Mal-CNN replaces

TABLE 1
Notation

Symbol	Meaning
x, L	Executable as a byte string, its length in bytes
$O(x)$	Protected version of x
$\mathbf{r}(x)$, $\mathbf{c}(x)$	Loader-relevant and descriptive header fields
$F(x)$, $\Gamma(x, x')$, $\Psi(x)$	Forged, false-flag, and edited versions of x
y^{d}, y, s	Detection label, group label, protection state
K, k	Number of groups, group index
$\mathcal{P}$, $\mathcal{U}$	Indices of paired and plain-only executables
$\kappa(x)$, q	Build key, fold index
$\mathcal{D}_{\mathrm{adv}}$, $\mathcal{T}$	Adversarial training set, tree ensemble of PIFA
$\mathbf{G}$, M	Global statistics view, number of its cells
$\mathbf{d}$	Byte-distribution view with 514 values
$\mathbf{H}$, $\mathbf{T}$, W	Head window, tail window, window length in bytes
N, N_v, D	Tokens per window, tokens of value view v, token dimension
$\mathbf{z}$, $\tilde{\mathbf{z}}$	Projected embeddings of x and $O(x)$
τ, α, β, ρ	Temperature, state weight, pair weight, forging probability

each convolution with a depth-wise separable convolution and adds a 7×7 spatial attention gate. In attribution, both read the contrast-enhanced images of 128×128 pixels released with that study. Our MalConv [16] reads the concatenated head and tail windows, embeds each byte, applies gated convolutions with kernel size and stride 512, and max-pools over time. The ImageNet-pretrained [37] CNNs ResNet-18 [38], MobileNetV2 [39], and EfficientNet-B0 [40] read the global statistics view in attribution. So do the Transformers Swin-T [41] and DeiT-Ti [42], which we call ViT-Ti hereafter. MobileNetV2 and EfficientNet-B0 enter only the detection experiment, and MalConv enters only the attribution experiment. The nearest-neighbor rule assigns a test sample the group of the training sample with the fewest differing loader-relevant field values. For each network run, we keep the epoch with the highest validation accuracy in detection or the highest validation macro-F_1 in attribution, with ties broken by validation accuracy. For the trees, we keep the boosting round with the lowest validation loss.

4 Method

Whole-file images average away the fine structure of a protected file, and byte sequences mix the regions that protection rewrites with those it keeps. The PE header keeps the group signal, but part of this signal lies in descriptive fields that an adversary can forge or copy from another group. A classifier on all header fields is accurate but follows these fields, whereas a classifier on the loader-relevant fields alone discards their signal and remains open to edits. Therefore, we propose the Perturbation-Invariant Forgery-Resistant Attribution (PIFA) model, which reads all header fields but trains on copies in which the fields that an adversary can change have been changed. We first build the datasets and splits in Section 4.1. Section 4.2 then defines the three manipulations and PIFA. Section 4.3 presents ObfusFormer.

4.1 Detection and Attribution Datasets Grouped by Build

We build one dataset for each task from the archive released with the study of Han *et al.* [13], as listed in Table 2. The training split of the detection dataset, which contains byte images derived from EMBER [15], holds 55,000 images. Of these images, 13,755 come from unlabeled EMBER samples that the original scripts stored in the benign folder. We remove these unlabeled images, which leaves 20,524 benign and 20,721 malicious training images. The test split of 2,454 benign and 2,546 malicious images, which the original scripts drew from the EMBER test set, stays unchanged. A fixed random 10% of the training images forms the validation set.

TABLE 2
Datasets

Detection, EMBER-derived			Attribution, APT			
Split	Benign	Malicious	Group	Plain	Prot.	Builds
Train	20,524	20,721	APT1	404	399	269
Test	2,454	2,546	APT10	228	223	213
Unlabeled*	13,755		APT28	162	160	154
			APT29	263	251	199
			APT30	164	157	145
			DarkHotel	269	260	218
			Energetic Bear	132	127	82
			Equation Group†	395	387	1
			Gorgon Group	345	284	323
			Winnti	385	326	287
Total	22,978	23,267	Used	2,352	2,187	1,890

The attribution dataset contains APT malware whose files are named by their SHA-256 hashes. The per-group file counts match the public collection of Boot *et al.* [3] except for one APT1 file. Han *et al.* studied ten of its groups and protected their PE files with VMProtect. We pair each protected file with its plain source through the SHA-256 hash of the source, which gives 2,747 plain and 2,574 protected executables. These executables have only 1,891 distinct build keys, and 1,095 of the 2,747 plain executables share their build key with another executable. All 395 Equation Group executables share one build key and one file size of 184,320 bytes, so a split grouped by build would place the whole group in one fold. We therefore exclude the Equation Group from the attribution experiments. This leaves 2,352 plain executables of $K = 9$ groups with 1,890 build keys, $|\mathcal{P}| = 2{,}187$, and $|\mathcal{U}| = 165$. We split them into five folds as defined in Section 3.1. In each fold, whole builds that hold about 10% of the training executables of each group form the validation set. On the split by time, the latest builds that hold about 10% of the remaining executables of each group form the validation set. Executables whose time stamp lies outside 2000 to 2020 always stay in training. The 474 test executables of this split, 404 of them with a protected version, were compiled from 2012 to 2018. No test build of a group is older than its training and validation builds with a valid time stamp. In Table 2, an asterisk marks the unlabeled EMBER images that we removed, and a dagger marks the Equation Group.

4.2 PIFA

PIFA reads the header fields $(\mathbf{r}(x), \mathbf{c}(x))$ of an executable with an ensemble $\mathcal{T}$ of gradient-boosted decision trees [18], as shown in Fig. 3. It trains $\mathcal{T}$ on the original samples and on three kinds of manipulated copies, which follow the threat model of Section 3.1. The forging function $F(\cdot)$ draws random values for every descriptive field, namely the time stamp, the linker, operating-system, and image versions, the checksum, the code and data sizes and bases, and the names of the first ten sections. It zeroes the symbol table, the certificate and debug directory entries, and the Rich header. It leaves every byte of a loader-relevant field unchanged, so $F(x)$ keeps every loader-relevant field of x. All changed bytes lie in the

Algorithm 1 Training of PIFA

Require: Training executables $\mathcal{X}$ with groups y, forgings $F^{(1)}$ and $F^{(2)}$, editing function Ψ, validation builds
Ensure: Tree ensemble $\mathcal{T}^*$
$\mathcal{D}_{\text{adv}} \leftarrow \emptyset$
for each $x \in \mathcal{X}$ with group y **do**
  Parse $\mathbf{r}(x)$ and $\mathbf{c}(x)$ from the PE header
  Draw a training executable x' of another group in the protection state of x
  Add (x, y), $(F^{(1)}(x), y)$, $(F^{(2)}(x), y)$, $(\Gamma(x, x'), y)$, and $(\Psi(x), y)$ to $\mathcal{D}_{\text{adv}}$ as in (3)
end for
Fit $\mathcal{T}$ on $\mathcal{D}_{\text{adv}}$ by gradient boosting with (4)
Stop after 100 rounds without a lower validation loss and return the best ensemble $\mathcal{T}^*$

first 1,088 bytes of a file. The false-flag function $\Gamma(x, x')$ gives x all descriptive fields of another executable x' and keeps the loader-relevant fields of x. The editing function $\Psi(\cdot)$ changes only loader-relevant fields whose values do not change how the program runs. It toggles, each with probability 0.5, the six characteristics flags that the PE specification marks as deprecated or informational for images and the terminal-server-aware DLL flag. It also sets the minor subsystem version to a random value from 0 to 9 when the major version is below 10. The function then draws new stack and heap reserve sizes from 64 KiB to 4 MiB and new commit sizes from 4 KiB to 64 KiB. Finally, it grows the virtual size of the last listed section within its section alignment, so that the image size stays unchanged. Forging changes header bytes, whereas false flags and edits change the parsed header fields.

For a set $\mathcal{X}$ of training executables, plain or protected, PIFA builds the adversarial training set

$$\mathcal{D}_{\text{adv}} = \bigcup_{x \in \mathcal{X}} \big\{(x, y),\ (F^{(1)}(x), y),\ (F^{(2)}(x), y),\ (\Gamma(x, x'), y),\ (\Psi(x), y)\big\}, \tag{3}$$

where y denotes the group of x, $F^{(1)}$ and $F^{(2)}$ denote two forgings with different random seeds, and x' denotes a random training executable of another group in the same protection state as x. PIFA then fits the trees by gradient boosting on

$$\mathcal{T}^* = \arg\min_{\mathcal{T}} \sum_{(x,y) \in \mathcal{D}_{\text{adv}}} \ell\big(\mathcal{T}(\mathbf{r}(x), \mathbf{c}(x)), y\big), \tag{4}$$

where ℓ denotes the cross-entropy of the softmax over the K group scores of $\mathcal{T}$, and $\mathcal{T}^*$ denotes the ensemble at the boosting round with the lowest loss on the validation builds. Each copy keeps the label y while it changes the descriptive fields or the editable loader-relevant fields. A split on a manipulated field therefore separates samples of one group and lowers the loss in (4) only where the field agrees with the unchanged fields. The trees are thus encouraged to keep a descriptive field where its value is consistent with the loader-relevant structure and to fall back on this structure where the field has been changed. Algorithm 1 summarizes the training of PIFA. The trees learn no features. The copies add no cost at inference time.

At test time, the manipulated test samples come from separate draws. We precompute nine forged copies of every executable with different random seeds, and the ninth forms the forged test set. In the same way, the ninth of nine edited copies forms the edited test set. For a false flag, every protected test executable receives all descriptive fields of a random protected training executable of another group. In detection, the trees read the 2,090 decoded feature values with the hyperparameters of the EMBER reference model [15].

4.3 ObfusFormer

ObfusFormer renders each executable into up to four views, tokenizes each view, and fuses the tokens of all views with one Transformer encoder, as shown in Fig. 4. The structure view $\mathbf{r} \in \mathbb{R}^{97}$ holds the loader-relevant header fields of Section 3.2, with zeros for absent sections. The byte-distribution view $\mathbf{d}$ summarizes the whole file without regard to byte order as

$$\mathbf{d} = [\mathbf{q}^{\text{b}};\ \mathbf{q}^{\text{e}};\ \log_2 L;\ \bar{e}_{\text{f}}] \in \mathbb{R}^{514}, \tag{5}$$

where $\mathbf{q}^{\text{b}} \in \mathbb{R}^{256}$ and $\mathbf{q}^{\text{e}} \in \mathbb{R}^{256}$ denote the byte histogram and the byte-entropy histogram of the file, both divided by L, and $\bar{e}_{\text{f}}$ denotes the mean Shannon entropy of its 256-byte chunks. The byte-entropy histogram counts the bytes of each chunk in a 16×16 table of its entropy bin and the high nibble of the byte value, in the manner of Saxe and Berlin [17]. The head window $\mathbf{H}$ holds the first $W = 65{,}536$ bytes, which contain the PE header, the section table, and the start of the sections. The tail window $\mathbf{T}$ holds the last W bytes, and both windows are zero-padded as a 256×256 image.

A value tokenizer turns a value view $v \in \{\mathbf{r}, \mathbf{d}\}$ into N_v tokens. Because header fields and histogram entries range over several orders of magnitude, it first compresses each value a to $\operatorname{sign}(a) \log(1 + |a|)$ and zero-pads the view to $16N_v$ values. For the structure view, it then encodes each value piecewise linearly over 16 bins whose edges are the quantiles of that field in the training samples [43]. We call this step the bin encoding. For the byte-distribution view, the tokenizer instead standardizes the values with Batch Normalization (BN). It splits the encoded values into chunks of 16 consecutive values and maps each chunk with its own linear layer as

$$\mathbf{E}_{v,g} = \mathbf{W}_{v,g} \tilde{\mathbf{v}}_g + \mathbf{b}_{v,g}, \quad g = 1, \ldots, N_v, \tag{6}$$

where $\mathbf{E}_{v,g} \in \mathbb{R}^D$ denotes token g of view v, $\tilde{\mathbf{v}}_g$ denotes the encoded values of chunk g, and $\mathbf{W}_{v,g}$ and $\mathbf{b}_{v,g}$ denote its weights and bias. The 97 and 514 values give $N_{\mathbf{r}} = 7$ and $N_{\mathbf{d}} = 33$ tokens.

A square image patch joins bytes one image width apart, whereas executable bytes follow their order. The byte-order tokenizer therefore flattens each window into a sequence of W bytes and embeds each byte value as $\boldsymbol{\phi}_b \in \mathbb{R}^{16}$. This gives the input $\mathbf{X}_v \in \mathbb{R}^{16 \times W}$ of window v, which the tokenizer maps to N tokens of consecutive bytes as

$$\mathbf{F}_v = \mathcal{S}_R \circ \cdots \circ \mathcal{S}_1\big(\text{GELU}(\text{BN}(\text{Conv}_{r_0}(\mathbf{X}_v)))\big), \tag{7}$$

where Conv_{r_0} denotes a one-dimensional convolution with stride $r_0 = 16$ and kernel $2r_0 + 1$, and GELU denotes the Gaussian Error Linear Unit. Each of the $R = 3$ stages $\mathcal{S}_r$ is a Depth-wise Separable Convolution (DSConv) with stride 4 and kernel 9, followed by BN and GELU, and we adapted it from Mal-CNN [13]. With $N = 64$, each token summarizes 1,024 consecutive bytes. An offset attention gate adapts the spatial attention of Mal-CNN to the offset axis. It multiplies $\mathbf{F}_v$ at each offset by the sigmoid of a convolution with kernel 7 over the channel-wise maximum and mean. A pointwise projection then gives the token sequence $\mathbf{E}_v \in \mathbb{R}^{N \times D}$ of window v.

We add to every token a learned positional embedding $\mathbf{P}_v$ of its view and a learned view embedding $\mathbf{u}_v \in \mathbb{R}^D$, and we prepend a class token $\mathbf{t}_{\text{cls}}$ to form

$$\mathbf{S}^{(0)} = [\mathbf{t}_{\text{cls}};\ \mathbf{E}_{v_1} + \mathbf{P}_{v_1} + \mathbf{u}_{v_1};\ \ldots;\ \mathbf{E}_{v_V} + \mathbf{P}_{v_V} + \mathbf{u}_{v_V}], \tag{8}$$

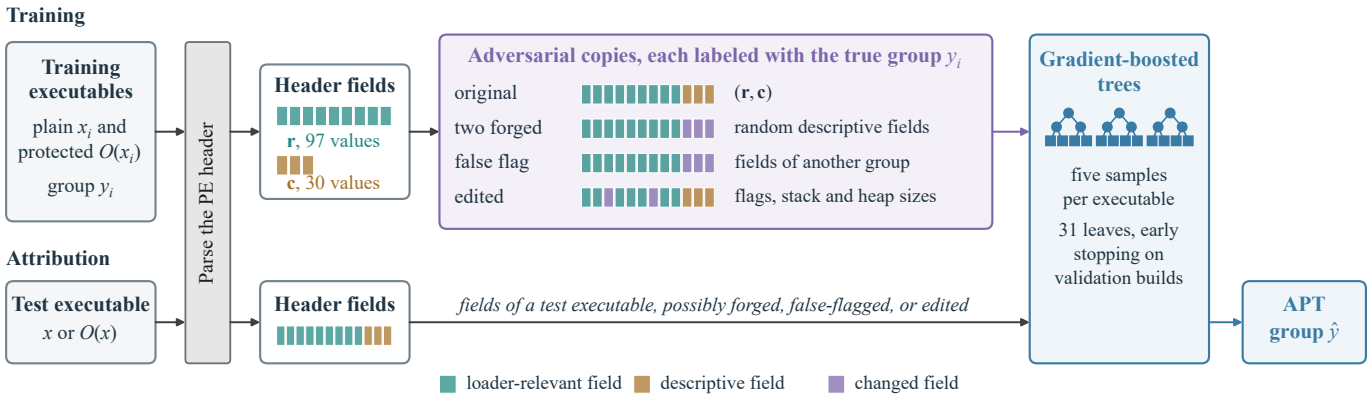


Fig. 3. Training and attribution with PIFA. One parser and one set of gradient-boosted trees serve both rows. During training, PIFA adds two forged copies, one false-flag copy, and one edited copy of every training executable to the original sample, all with the true group. At test time, the trees read the header fields of a test executable, which an adversary may have forged, copied from another group, or edited. Teal, gold, and purple cells mark loader-relevant, descriptive, and changed fields.

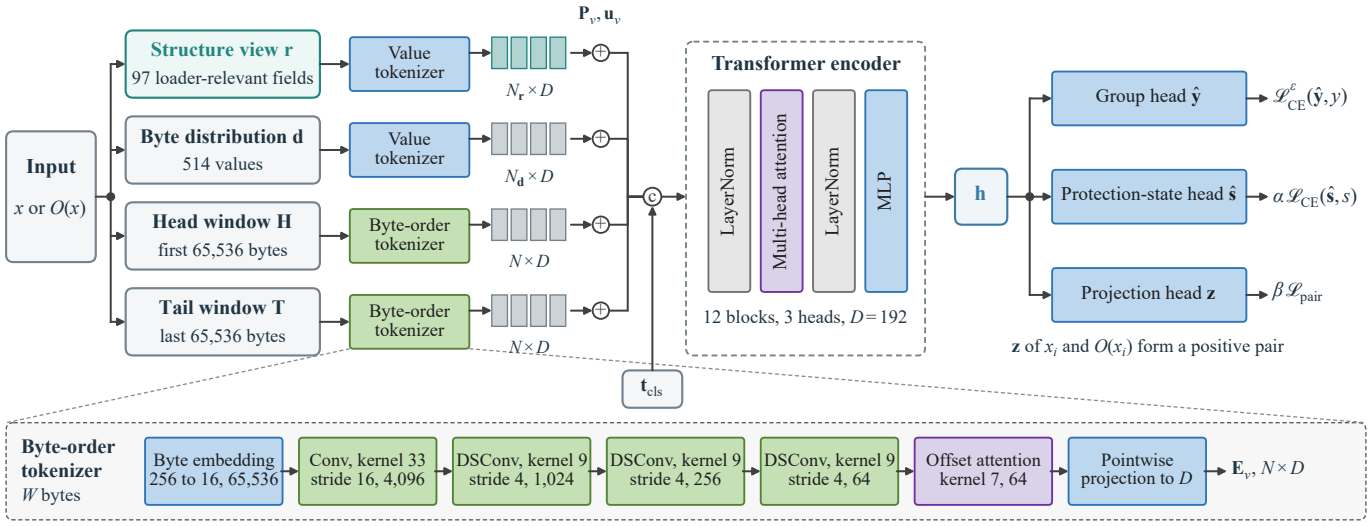


Fig. 4. Architecture of ObfusFormer. Value tokenizers embed the structure view and the byte-distribution view. Each byte-order tokenizer flattens a window in byte order and converts it into N tokens. The tokens of all views, with positional and view embeddings, enter a shared Transformer encoder together with a class token. The class-token embedding $\mathbf{h}$ feeds the group head, the protection-state head, and the projection head of the obfuscation-paired loss. The zoomed panel gives the layers of one byte-order tokenizer with the channel counts and sequence lengths of their outputs.

where $\mathbf{S}^{(0)}$ denotes the input sequence of the encoder and $v_1, \ldots, v_V$ denote the V views that a configuration reads. The encoder has the structure of ViT-Ti [42]. It stacks 12 pre-norm blocks of three-head self-attention [44] and a Multilayer Perceptron (MLP) with $D = 192$. We initialize it from ImageNet-pretrained ViT-Ti, while the tokenizers and embeddings train from scratch. The normalized class token after the last block gives the embedding $\mathbf{h} \in \mathbb{R}^D$. This embedding feeds a linear group head $\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{W}_y \mathbf{h} + \mathbf{b}_y)$, a linear protection-state head $\hat{\mathbf{s}}$, and a two-layer projection head $\mathbf{z} = \pi(\mathbf{h})$, which is used only during training. For detection, ObfusFormer reads the 2,090 decoded feature values as one value view with $N_v = 131$ tokens, and the group head becomes a two-class detection head.

A protected executable and its plain source carry the same group label but share few byte statistics. We treat each training pair as two versions of one executable and pull their projected embeddings together, in the manner of contrastive learning on augmented views [14] with the information noise-contrastive estimation loss [45]. Each mini-batch contains B executables drawn without replacement. Every executable i in $\mathcal{P}$ enters the batch with both x_i and $O(x_i)$, and every executable in $\mathcal{U}$ enters with x_i only. The obfuscation-paired loss over the batch is

$$\mathcal{L}_{\mathrm{pair}} = -\frac{1}{2B_p} \sum_{i=1}^{B_p} \Bigg[\log \frac{\exp(\langle \mathbf{z}_i, \tilde{\mathbf{z}}_i \rangle / \tau)}{\sum_j \exp(\langle \mathbf{z}_i, \tilde{\mathbf{z}}_j \rangle / \tau)} + \log \frac{\exp(\langle \tilde{\mathbf{z}}_i, \mathbf{z}_i \rangle / \tau)}{\sum_j \exp(\langle \tilde{\mathbf{z}}_i, \mathbf{z}_j \rangle / \tau)} \Bigg], \tag{9}$$

where $\mathcal{L}_{\mathrm{pair}}$ denotes the paired loss, $B_p \le B$ denotes the number of executables of the batch that have a protected version, and $\mathbf{z}_i$ and $\tilde{\mathbf{z}}_i$ denote the ℓ_2-normalized projections of x_i and $O(x_i)$, respectively. Further, $\langle \cdot, \cdot \rangle$ denotes the inner product, j runs over the B_p pairs of the batch, and τ denotes the temperature. The other pairs of the batch, including pairs of the same group, act as negatives. The full objective is

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}}^{\epsilon}(\hat{\mathbf{y}}, y) + \alpha \, \mathcal{L}_{\mathrm{CE}}(\hat{\mathbf{s}}, s) + \beta \, \mathcal{L}_{\mathrm{pair}}, \tag{10}$$

Algorithm 2 Training of ObfusFormer

Require: Training indices $\mathcal{P} \cup \mathcal{U}$, executables x_i, protected versions $O(x_i)$, groups y_i, protection states s_i, forged copies $F^{(1)}(\cdot), \ldots, F^{(8)}(\cdot)$, validation set, number of epochs n_{ep}, probability ρ
Ensure: Trained parameters θ^*
Initialize the encoder from ViT-Ti and the tokenizers, embeddings, and heads randomly, and fit the bin edges of the structure view on the training samples
for $t = 1$ **to** n_{ep} **do**
  for each batch $\mathcal{B}$ of B indices from $\mathcal{P} \cup \mathcal{U}$ **do**
    Collect x_i for $i \in \mathcal{B}$ and $O(x_i)$ for $i \in \mathcal{B} \cap \mathcal{P}$
    Replace each sample with a random forged copy with probability ρ
    Render the views **r**, **d** with (5), **H**, and **T** of the configuration
    Tokenize **r** and **d** with (6) and the windows with (7) and the offset attention gate
    Encode the sequence (8) and read $\mathbf{h}$, $\hat{\mathbf{y}}$, $\hat{\mathbf{s}}$, and $\mathbf{z}$
    Compute $\mathcal{L}_{\text{pair}}$ over the pairs in $\mathcal{B}$ with (9)
    Update θ with AdamW on $\mathcal{L}$ in (10)
  end for
  Compute the validation macro-F_1 with (2), and if it is the best so far, set $\theta^* \leftarrow \theta$
end for
return θ^*

where $\mathcal{L}$ denotes the training loss, y and s denote the group and protection state of a sample, and $\mathcal{L}_{\text{CE}}^{\epsilon}$ and $\mathcal{L}_{\text{CE}}$ denote the batch-averaged cross-entropy with and without label smoothing ϵ [46], respectively. The weights α and β scale the protection-state loss and the paired loss. Under the Plain and VM protocols, which have no pairs and one protection state, we set $\alpha = \beta = 0$.

The head window holds the descriptive header fields at fixed offsets, so a byte-order tokenizer can learn their values. To keep the configurations of ObfusFormer that read a window from relying on these fields, we train them also on the forged copies of Section 4.2. Because all changed bytes lie in the first 1,088 bytes, we update the other views exactly. We recompute the byte histogram, the byte-entropy histogram, and the mean chunk entropy from the changed 256-byte chunks, and we write the changed bytes into the head window. For the image networks, we update the cells of **G** that cover the changed bytes. We replace each training sample with one of eight forged copies, chosen at random, with probability ρ. A variant of ObfusFormer that we use in the tests of edits also trains on the other eight edited copies. Algorithm 2 summarizes the training of ObfusFormer.

5 Experiment

5.1 Experimental Setup

We ran the experiments on a laptop with an Intel Core i5-12450H Central Processing Unit (CPU), 16 GB of memory, and an NVIDIA GeForce RTX 4050 Laptop Graphics Processing Unit (GPU) with 6 GB of memory. The laptop ran Windows 11 with Python 3.11.15, PyTorch 2.11.0 with CUDA 12.6, timm 1.0.30, scikit-learn 1.8.0, and LightGBM 4.6.0. A second machine had an Intel Core i7-14650HX CPU and an NVIDIA GeForce RTX 4070 Ti SUPER GPU with 16 GB of memory. It ran Ubuntu 24.04 with Python 3.12.13, PyTorch 2.11.0 with CUDA 12.8, and the same timm version. On the second machine, we ran the attribution runs of the networks and of the configurations of ObfusFormer that read more than the structure view. The second machine also ran the detection runs of all networks with seed 2 and of MobileNetV2 and EfficientNet-B0 with seed 1. The detection runs of Mal-CNN and the CNN with seed 0 and the Mal-CNN runs on the released augmented images ran on the laptop CPU in 32-bit precision. All trees ran on the CPU of the laptop. We trained all networks with shared hyperparameters and used method-specific values only where Table 3 marks them. On the GPU, we used bfloat16 autocasting. The optimizer was AdamW [47] with a cosine learning-rate schedule, one warm-up epoch, and gradient-norm clipping at 1.0. In attribution, all trees, PIFA included, had 31 leaves, at least 10 samples per leaf, a learning rate of 0.05, a feature fraction of 0.5, and bagging of 80% of the samples in each round. Boosting stopped after 100 of at most 2,000 rounds without an improvement of the validation loss.

TABLE 3
Training hyperparameters of the networks

Item	Detection	Attribution
Epochs	8	15
Batch in samples	32	16, Joint: 8 executables
Learning rate	3×10^{-4}	3×10^{-4}
CNN, Mal-CNN, MalConv	5×10^{-4}	5×10^{-4}
Weight decay, stochastic depth†	0.05, 0.1	0.05, 0.1
Label smoothing ϵ	0	0.1
ObfusFormer value tokens	131	7 for **r**, 33 for **d**
ObfusFormer window tokens N, bins of **r**		64, 16
τ, α, β, ρ		0.1, 0.2, 0.5, 0.5
Swin-T gradient-accumulation steps	1	4

The dagger in Table 3 marks ViT-Ti, Swin-T, EfficientNet-B0, and the encoder of ObfusFormer. We selected the views of ObfusFormer on the validation splits of the Joint protocol. Among the six view configurations, the structure view alone had the highest mean validation macro-F_1, 88.3%, against 79.0% for the byte-distribution view with the two windows. In the comparisons, ObfusFormer denotes the configuration with the structure view alone. ObfusFormer-B denotes the configuration with the byte-distribution view and the two windows, which reads the same inputs as the byte-level methods. For detection, value tokens standardized with BN had the highest validation accuracy among five tokenizations with seed 0, 94.3%, against 93.6% for the bin encoding.

5.2 Experiment 1: Malware Detection

We trained each detection method with three seeds on the cleaned training images and evaluated it on the 5,000 test images. Table 4 lists the results, and Fig. 5 shows the receiver operating characteristic curves. The gradient-boosted trees on the decoded feature values detected malware most accurately, with an accuracy of 94.7% and a ROC-AUC of 0.988. At an FPR of 1%, they detected 82.9% of the malicious test images. ObfusFormer, which reads the same values, had an accuracy of 91.5%, which is 3.2 points below the trees. It was 0.6 points above the CNN, the most accurate image network with 90.9%. With the unlabeled samples kept in the benign class, as in the original scripts, Mal-CNN had an accuracy of 86.3% with seed 0, against 88.1% on the cleaned images. Models on the decoded feature values thus detected malware more accurately than the networks on the images rendered from them.

TABLE 4
Detection on the EMBER-derived test images, means over three seeds

Method	Acc. (%)	F_1 (%)	ROC-AUC	TPR at 1% (%)
Networks of Han et al. [13]				
CNN	90.9	91.3	0.971	70.0
Mal-CNN	88.5	88.8	0.957	62.9
ImageNet-pretrained networks				
MobileNetV2	89.4	89.7	0.960	64.5
ResNet-18	89.6	90.0	0.963	63.2
EfficientNet-B0	90.1	90.4	0.964	61.8
ViT-Ti	90.2	90.5	0.965	65.9
Swin-T	90.7	90.9	0.970	68.7
Gradient-boosted trees				
Decoded feature values	**94.7**	**94.8**	**0.988**	**82.9**
Transformer of this work				
ObfusFormer	91.5	91.7	0.971	72.0

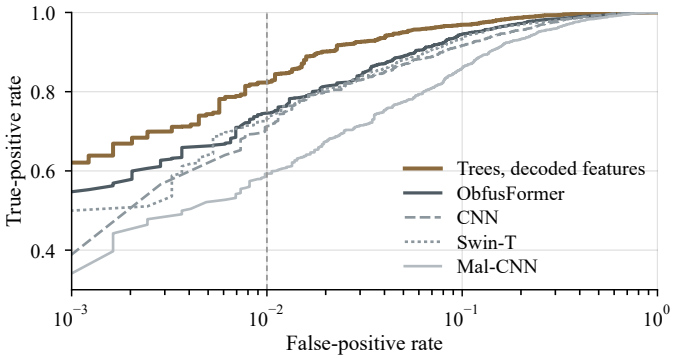


Fig. 5. Detection Receiver Operating Characteristic curves on the EMBER-derived test images with seed 0. The dashed line marks a false-positive rate of 1%.

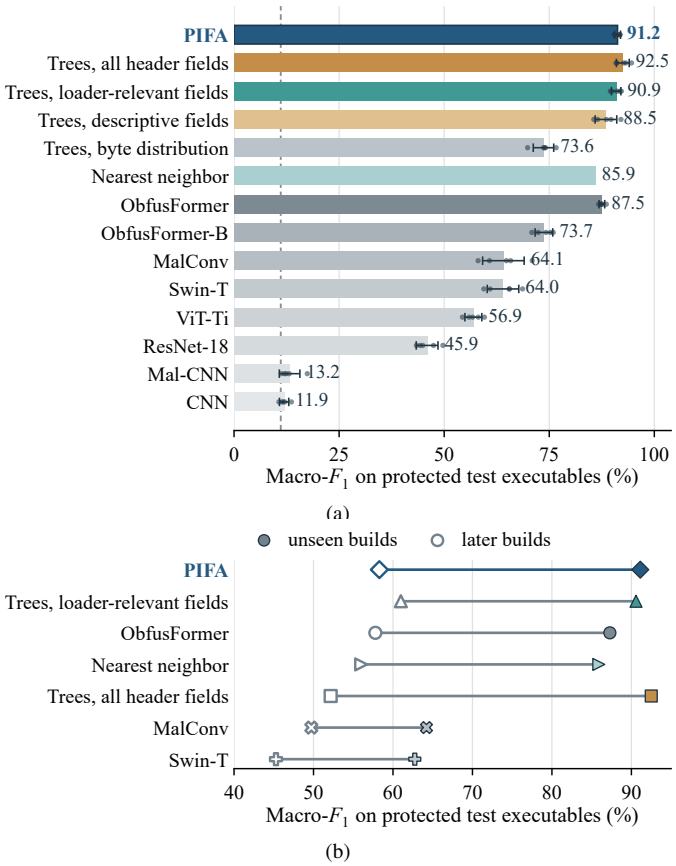


Fig. 6. Attribution of protected executables under the Joint protocol. (a) Macro-F_1 on the protected test executables of the folds grouped by build with seed 0. Bars give means over the five folds, caps the standard deviation, and dots the folds. The dashed line marks a uniformly random classifier of nine groups. (b) Macro-F_1 on unseen builds of the grouped folds and on later builds of the split by time, both averaged over seeds 0 to 2. The nearest-neighbor rule has one run per split.

5.3 Experiment 2: Attribution Under VMProtect and Header Manipulation

We first measured how the released split of the protected-image dataset affected accuracy. Over three seeds, Mal-CNN had a mean accuracy of 81.2% on the released split, in which 820 of the 1,015 test executables also contributed images to training. A single executable contributed as many as 613 augmented images to the released set. On a split of the same images grouped by source executable and with the same training settings, the accuracy of our re-implementation fell to 16.5%, and its macro-F_1 to 18.3%. Builds also repeat in the corpus. With folds stratified only by group, 1,020 of the 2,747 plain executables had their build in the training part of another fold. We therefore report all other attribution results on the folds grouped by build and on the split by time.

Table 5 lists the macro-F_1 of every method under the three training protocols, with means and standard deviations over the folds and seeds. In its column heads, an arrow leads from the training protocol to the version of the test executables, and an asterisk marks protected test executables with forged descriptive fields. On unseen builds, PIFA attributed the protected test executables with a macro-F_1 of 91.2% under the Joint protocol, as shown in Fig. 6(a). Over seeds 0 to 2, its macro-F_1 was 91.1%. The clean-trained trees on all header fields were more accurate on these untouched samples, with 92.5%, and they were more accurate than PIFA in each of the five folds. The trees on the loader-relevant fields alone had 90.9%. ObfusFormer, which reads the same loader-relevant fields, had 87.5%, and the nearest-neighbor rule on these fields had 85.9%. The methods that read images or bytes ranked lower. ObfusFormer-B had 73.7%, and the baseline networks had at most 64.1%, the value of MalConv. The CNN of Han *et al.* stayed at 11.9%, near the 11.1% of a uniformly random classifier of nine groups. A paired bootstrap over the protected test executables confirmed that the loader-relevant trees exceeded ObfusFormer and that ObfusFormer exceeded the most accurate baseline network. ObfusFormer has 5.7 million parameters and trained in 2.0 minutes per fold on the laptop GPU. PIFA trained in about 4 seconds per fold on the laptop CPU, including the generation of the copies.

Under the Plain protocol, only the clean-trained trees that read descriptive fields attributed protected executables above 40%. Trained on plain executables alone, PIFA had 12.4% on protected ones, against 43.6% for the clean-trained trees on all header fields and 65.2% for the trees on the descriptive fields alone. VMProtect keeps the descriptive fields but changes the entry point, the image size, and the section count, so a model trained on plain executables transfers mainly through the descriptive fields. PIFA is trained to discount these fields. It therefore needs protected training executables. The group signal of the loader-relevant fields lay mainly in the layout of a program. Shuffling the section-table fields lowered the protected-test macro-F_1 of the loader-relevant trees by 49.5 points, and shuffling the data directories lowered it by 28.9 points. These two layout groups carried 81.9% of the split

TABLE 5
Attribution macro-F_1 of APT executables in percent, with PIFA in bold

	Folds grouped by build						Later builds	
	Seed 0					Seeds 0 to 2	Seeds 0 to 2	
Method	Plain→VM	VM→VM	Joint→Plain	Joint→VM	Joint→VM∗	Joint→VM	Joint→Plain	Joint→VM
Networks of Han et al. [13]								
CNN	5.6±2.1	15.2±3.8	75.0±3.3	11.9±1.1		12.0±1.1	45.3±1.3	13.6±0.5
Mal-CNN	5.0±1.3	16.9±4.0	73.4±3.2	13.2±2.4		14.5±3.0	43.1±1.2	12.7±2.8
Other networks								
MalConv	7.8±1.6	61.5±5.8	82.3±2.3	64.1±4.9	59.2±5.6	64.2±3.9	51.3±2.3	49.7±1.5
ResNet-18	3.9±0.5	41.5±1.4	83.7±2.2	45.9±2.6	45.9±2.6	46.1±2.3	50.9±1.0	34.5±0.6
ViT-Ti	6.0±1.6	58.3±2.4	81.5±2.4	56.9±2.0	57.2±1.9	57.9±3.5	50.9±2.0	40.3±3.1
Swin-T	5.5±1.1	62.9±2.5	84.1±2.2	64.0±3.8	64.1±3.8	62.7±3.3	53.2±4.5	45.3±1.6
Gradient-boosted trees								
PIFA	**12.4±4.2**	**89.8±1.1**	**91.5±1.4**	**91.2±0.7**	**91.1±1.1**	**91.1±1.1**	**58.5±1.8**	**58.3±1.5**
All header fields	43.6±4.2	92.4±1.1	92.8±1.5	92.5±1.5	48.3±3.4	92.5±1.7	50.3±1.2	52.2±0.6
Loader-relevant fields	16.3±3.4	89.5±1.1	91.0±1.5	90.9±1.1	90.9±1.1	90.6±1.3	59.6±0.3	61.0±0.6
Descriptive fields	65.2±1.8	87.9±2.2	89.1±3.3	88.5±2.6	9.0±2.2	88.4±2.5	27.8±3.0	22.5±3.7
Byte distribution	4.6±1.8	74.0±3.8	88.3±1.2	73.6±2.4	68.6±2.8	72.9±2.7	56.0±0.4	46.3±0.3
Nearest neighbor								
Loader-relevant fields			88.5±0.9	85.9±2.8	85.9±2.8		55.2	56.0
Transformers of this work								
ObfusFormer	10.5±7.3	86.9±1.5	87.6±1.5	87.5±0.7	87.5±0.7	87.3±1.4	57.3±0.8	57.8±1.8
ObfusFormer-B	8.2±3.0	60.8±3.7	80.9±3.8	73.7±2.1	73.7±2.0	70.0±8.1	52.9±6.2	50.1±4.3

TABLE 6
Macro-F_1 under header manipulation and share attributed to the donor group in percent, with PIFA in bold

Method	Original	Forged	Edited	False flag	Donor
Gradient-boosted trees					
PIFA	**91.2**	**91.1**	**91.0**	**90.9**	**1.6**
All header fields	92.5	48.3	91.8	50.6	29.3
Loader-relevant fields	90.9	90.9	85.3	90.9	1.2
Train also on edited copies.	90.6	90.6	90.3	90.6	1.7
Descriptive fields	88.5	9.0	88.5	0.0	100.0
Nearest neighbor					
Loader-relevant fields	85.9	85.9	84.6	85.9	
Transformer of this work					
ObfusFormer	87.5	87.5	68.3	87.5	
Train also on edited copies.	87.2	87.2	86.8	87.2	

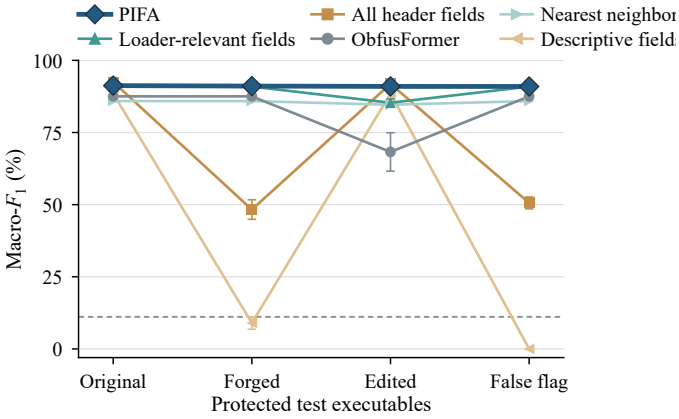


Fig. 7. Macro-F_1 on the protected test executables under header manipulation, with the Joint protocol on the folds grouped by build and seed 0. Markers give means over the five folds and bars the standard deviation. The structure view of ObfusFormer and the nearest-neighbor rule hold no descriptive field, so forging and false flags leave their predictions unchanged. The dashed line marks a uniformly random classifier of nine groups.

gain of the trees. The toolchain fields, which are the machine, the alignments, the image base, and the stack and heap sizes, mattered far less, as did the coarse sizes. The trees on the layout fields alone had a protected-test macro-F_1 of 90.0%, against 58.3% on the toolchain fields alone and 80.1% on the coarse file sizes and section count alone. Trained on plain executables, trees on the 49 to 52 header fields that VMProtect kept in at least 95% of the training pairs had 89.1% on protected executables. Trees on the 35 to 37 kept loader-relevant fields alone had 84.3%.

Table 6 and Fig. 7 give the macro-F_1 on the manipulated protected test executables under the Joint protocol with seed 0. The donor column gives the share of the false-flagged samples that a method attributed to the group of the donor executable. Forging the descriptive header fields lowered the macro-F_1 of the clean-trained trees on all header fields from 92.5% to 48.3%. The trees on the descriptive fields alone fell from 88.5% to 9.0%, below the 11.1% of a uniformly random classifier. False flags redirected the attribution of these trees. With the descriptive fields of a training executable of another group, the trees on all header fields had a macro-F_1 of 50.6% and attributed 29.3% of the protected test executables to the group of the donor. The trees on the descriptive fields alone attributed every false-flagged sample to the donor group. Edits of the loader-relevant fields lowered the macro-F_1 of the methods that read these fields. The trees on the loader-relevant fields fell from 90.9% to 85.3%, the nearest-neighbor rule from 85.9% to 84.6%, and ObfusFormer from 87.5% to 68.3%. Training on edited copies limited the loss to 0.3 points for the loader-relevant trees and to 0.4 points for ObfusFormer.

PIFA resisted all three manipulations. Its macro-F_1 was 91.1% under forging, 91.0% under editing, and 90.9% under a false flag, against 91.2% on the original samples. PIFA attributed 1.6% of the false-flagged samples to the donor group, close to the 1.2% of the loader-relevant trees, which read no descriptive field. Its worst case over the three manipulations, 90.9%, was the highest of the compared methods. The loader-relevant trees trained with edited

copies, which apply the idea of PIFA to a subset of the fields, came closest with a worst case of 90.3%. PIFA exceeded them by 0.7 points on the original samples and had the higher worst case in three of the five folds. Against the clean-trained loader-relevant trees, the worst case of PIFA was higher in all five folds. Forging also lowered the macro-F_1 of the byte-level methods that read the header bytes. MalConv fell from 64.1% to 59.2% and the byte-distribution trees from 73.6% to 68.6%. ObfusFormer-B kept 73.7% when trained with forged copies but fell from 71.2% to 61.0% without them. The image networks changed by at most 0.3 points, because forging changes only the first 1,088 bytes of a file.

On the split by time, every method except the CNN of Han *et al.* attributed the later builds less accurately than the unseen builds of the grouped folds, as shown in Fig. 6(b). The CNN stayed near the level of a random classifier on both. The trees on the loader-relevant fields had a protected-test macro-F_1 of 61.0%, which is 29.6 points below their value on the grouped folds. PIFA had 58.3%, which is 2.7 points below the loader-relevant trees. ObfusFormer had 57.8%, the clean-trained trees on all header fields 52.2%, and MalConv, the most accurate baseline network, 49.7%. The nearest-neighbor rule had 56.0%, or 65.4% when the validation builds, which are the latest builds before the test builds, joined its references. These results suggest that the accuracy of all methods on later builds depends on how close in time their training builds are.

5.4 Ablation

We removed one type of adversarial copy from PIFA at a time and retrained it under the Joint protocol on all five folds with seed 0. Each variant shares the code path, folds, test samples, and tree settings of PIFA. Fig. 8(a) gives the macro-F_1 of each variant on the original and manipulated test samples. The edited copies had the largest effect. Without them, PIFA fell from 91.0% to 83.1% under editing, below the 85.3% of the clean-trained loader-relevant trees. A possible reason is that training on forged and false-flag copies alone shifts the trees toward the loader-relevant fields, which editing then attacks. Removing the forged copies or the false-flag copies lowered the worst case by a small amount, to 89.7% and 90.1%, respectively. Both types of copy change the descriptive fields. With either type alone, PIFA lost at most 1.2 points against its full version under forging and false flags. Without any copy, PIFA reduces to the clean-trained trees on all header fields, with 92.5% on the original samples but 48.3% under forging.

For ObfusFormer, we changed one design choice at a time and compared each variant with the configuration that it was derived from on the same folds and seeds. Table 7 lists the views and the training variants, with seed 0 on the validation, plain, protected, and forged protected test samples and with seeds 0 to 2 on the protected test samples. As in Table 5, an asterisk marks forged descriptive fields. The structure view alone gave the highest protected-test macro-F_1, 87.5% with seed 0. Adding the two windows lowered it to 78.7%, and adding the windows and the byte distribution lowered it to 77.9%. Without the structure view, the byte views had at most 73.7%. Over seeds 0 to 2, the bin encoding mattered most, as shown in Fig. 8(b). Standardization instead of the bin encoding changed the protected-test macro-F_1 of ObfusFormer by −7.3 points, and random initialization of the encoder changed it by −1.7 points. Removing the paired loss or the protection-state head changed it by +0.2 and −0.5 points, which lie within the range of the paired runs. For ObfusFormer-B, the forged copies helped only on forged samples, where their removal changed the macro-F_1 by −9.6 points.

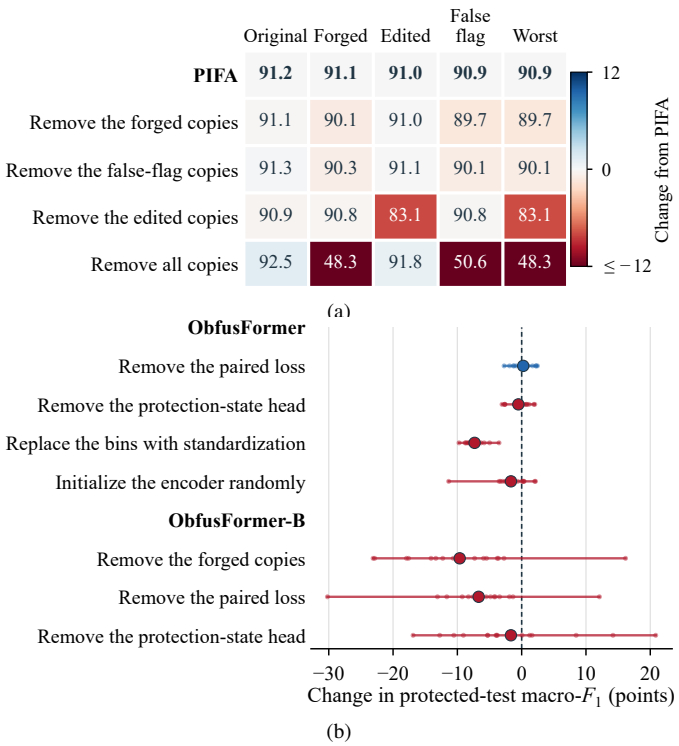


Fig. 8. Ablations. (a) Macro-F_1 of PIFA without one type of adversarial copy or without all copies, on the original and manipulated protected test executables with seed 0. The worst column gives the lowest value over the three manipulations, and the color gives the change from PIFA. (b) Change in the protected-test macro-F_1 of ObfusFormer and ObfusFormer-B when one part of the training is removed or replaced, over the 15 paired runs of seeds 0 to 2 and five folds. Dots give means and segments the range of the runs. The removal of the forged copies is scored on the forged test samples.

TABLE 7
Views and training of ObfusFormer, macro-F_1 in percent

	Seed 0				Seeds 0 to 2
Configuration	Val.	Plain	Prot.	Prot.∗	Prot.
Views					
r	88.3	87.6	87.5	87.5	87.3±1.4
d	57.0	84.5	16.9	16.8	16.9±3.4
H, **T**	72.1	72.5	67.6	67.4	68.8±7.2
d, **H**, **T**	79.0	80.9	73.7	73.7	70.0±8.1
r, **H**, **T**	83.7	82.6	78.7	78.7	76.5±2.8
r, **d**, **H**, **T**	83.1	83.9	77.9	77.9	77.4±3.4
Training					
ObfusFormer	88.3	87.6	87.5	87.5	87.3±1.4
Remove the paired loss.	89.0	88.5	87.9	87.9	87.5±1.3
Remove the protection-state head.	87.3	86.5	86.7	86.7	86.8±1.8
Replace the bins with standardization.	81.3	81.5	80.0	80.0	80.0±1.6
Initialize the encoder randomly.	86.7	85.8	85.3	85.3	85.6±2.5
ObfusFormer-B	79.0	80.9	73.7	73.7	70.0±8.1
Remove the forged copies.	78.3	80.7	71.2	61.0	72.0±5.5
Remove the paired loss.	73.0	77.5	65.0	64.9	63.3±6.0
Remove the protection-state head.	73.5	75.7	64.6	64.3	68.3±4.8

6 Discussion

6.1 Robustness Through Adversarial Copies

The permutation results indicate that the group signal lies mainly in the parts of the PE header that virtualization preserves. VMProtect protected the code of a program but kept the original section table and data directories in the header, although it appended sections and moved the entry point. These fields carried 81.9% of the split gain of the loader-relevant trees, so the surviving signal is largely how a group lays out its programs. This signal is in part a fingerprint of related builds. On builds compiled after the training builds, the loader-relevant trees were 29.6 points less accurate than on the folds grouped by build. On the grouped folds, the nearest-neighbor rule on the same fields came within 1.6 points of ObfusFormer. On the split by time, it improved when the latest earlier builds joined its references. PIFA therefore attributes a new sample most reliably when related builds of its group are available. A split that ignores builds overstates this accuracy.

The descriptive header fields add a real but forgeable signal. The clean-trained trees on all header fields were more accurate than the loader-relevant trees on untouched samples. Forging these fields or copying them from another group, however, lowered the macro-F_1 of these trees by more than 40 points. With copied fields, the trees sent 29.3% of the samples to the donor group of a false flag. PIFA keeps part of the descriptive signal without this vulnerability. Because every copy keeps the true group, a split on a manipulated field gains little. The trees then rely on the fields that the manipulations leave unchanged. In the ablation, removing all copies lowered the macro-F_1 under forging from 91.1% to 48.3%. Once the copies are in place, the descriptive fields add a small gain of 0.7 points over the loader-relevant trees trained with edited copies. Against the clean-trained trees, PIFA gives up 1.3 points on untouched samples. Adversarial copies thus trade a small loss of clean accuracy for robustness to the three manipulations that we tested, at no cost at inference time.

A learned representation did not match the header-field trees. On the protected test executables, ObfusFormer was 23.4 points above the most accurate baseline network but below the header-field trees, and standardization instead of the bin encoding widened the gap. Gorishniy *et al.* [43] reported that piecewise-linear encodings of numerical features improved Transformers on tabular data. Grinsztajn *et al.* [48] reported that tree ensembles remained more accurate than deep networks, including a Transformer, on medium-sized tabular data. The header fields are such tabular data.

6.2 Remaining Limitations

All protected executables came from one protector, VMProtect, which Han *et al.* [13] applied with options that their paper does not report. The archive of their study contains version 3.5.0. Other virtualizers such as Themida and Code Virtualizer, and protector options that pack the output file, may rebuild the section table and remove the layout that the loader-relevant fields describe. PIFA also needs protected training executables, because adversarial training discounts the descriptive fields through which a model trained on plain executables transfers. We forged header bytes but changed the parsed fields for false flags and edits. We did not write or execute any manipulated file. The edits therefore assume that the changed flags and stack and heap sizes leave a program runnable. Larger functionality-preserving changes, such as injected or padded sections [30], [31], would alter the layout fields and lie outside our tests. An adaptive adversary who knows PIFA could also search for edits of the layout fields that keep a program runnable. The split by time orders builds by their compile time stamps, which an author can set. The APT labels come from indicators in threat-intelligence reports, and Gray *et al.* [4] noted a lack of assurance about how such labels were assigned. The detection images render EMBER feature vectors rather than raw files, so the detection experiment did not test the structure view or the windows. These data also contain no protected benign software, so the effect of protection alone on a detection verdict remains unmeasured.

6.3 Directions for Robust Attribution

Protecting the training executables with several virtualizers and option sets would show whether loader-relevant header fields still identify a group when the protector rebuilds the header. Writing the manipulated headers back into the files and running them in a sandbox would confirm that the copies of PIFA describe runnable programs. Injecting and padding sections of the test executables would test attribution against layout edits, and the same edits could form a fourth type of adversarial copy. Dating builds by the time they were first seen rather than by their compile time stamps would remove the dependence of the split by time on a forgeable field.

7 Conclusion

We have proposed the Perturbation-Invariant Forgery-Resistant Attribution (PIFA) model, which reads every field of the PE header with gradient-boosted trees. PIFA trains these trees on forged, false-flag, and edited copies of each executable, all labeled with the true group. On unseen builds of nine APT groups, PIFA has attributed VMProtect-protected executables with a macro-F_1 of 91.2%. Under forging, false flags, and edits, its macro-F_1 has stayed at 90.9% or higher, the highest worst case of the compared methods. The clean-trained trees on all header fields have been 1.3 points more accurate on untouched samples but have fallen to 48.3% under forging. In the ablation, removing the edited copies has cost 7.9 points under edits, whereas removing the forged or the false-flag copies has cost at most 1.2 points under any manipulation. ObfusFormer, the multi-view Transformer that we have developed, has attributed protected executables with a macro-F_1 of 87.5% from the structure view alone, above every image and byte network. On later builds, the macro-F_1 of PIFA has fallen to 58.3%, which indicates that its accuracy rests in part on related builds in training.

In our future work, we will protect executables with several virtualizers and option sets. We will also write the manipulated headers back into runnable files to test whether adversarial copies keep the attribution robust when the protector rebuilds the header.

References

[1] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *Journal of Network and Computer Applications*, vol. 153, p. 102526, 2020.

[2] I. Rosenberg, G. Sicard, and E. David, "DeepAPT: Nation-state APT attribution using end-to-end deep neural networks," in *Artificial Neural Networks and Machine Learning – ICANN 2017*, ser. Lecture Notes in Computer Science, vol. 10614. Cham, Switzerland: Springer, 2017, pp. 91–99.

[3] C. Boot, A. C. Serban, and E. Poll, "Supervised learning for state-sponsored malware attribution," Manuscript, Radboud University, Nijmegen, The Netherlands, 2019, accessed: Oct. 11, 2026. [Online]. Available: https://cs.ru.nl/~aserban/papers/pdf/MalwareStateAttribution.pdf

[4] J. Gray, D. Sgandurra, L. Cavallaro, and J. Blasco Alis, "Identifying authorship in malicious binaries: Features, challenges & datasets," *ACM Computing Surveys*, vol. 56, no. 8, pp. 1–36, 2024.

[5] S. Li, C. Jia, P. Qiu, Q. Chen, J. Ming, and D. Gao, "Chosen-instruction attack against commercial code virtualization obfuscators," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2022.

[6] P. Kochberger, S. Schrittwieser, S. Schweighofer, P. Kieseberg, and E. Weippl, "SoK: Automatic deobfuscation of virtualization-protected applications," in *Proc. 16th Int. Conf. Availability, Reliability and Security (ARES)*, Vienna, Austria, 2021, pp. 1–15.

[7] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: Visualization and automatic classification," in *Proc. 8th Int. Symp. Visualization for Cyber Security (VizSec)*. Pittsburgh, PA, USA: ACM, 2011, pp. 1–7.

[8] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang, and F. Iqbal, "Malware classification with deep convolutional neural networks," in *Proc. 9th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, 2018, pp. 1–5.

[9] H. Yakura, S. Shinozaki, R. Nishimura, Y. Oyama, and J. Sakuma, "Malware analysis of imaged binary samples by convolutional neural network with attention mechanism," in *Proc. 8th ACM Conf. Data and Application Security and Privacy (CODASPY)*, Tempe, AZ, USA, 2018, pp. 127–134.

[10] S. Seneviratne, R. Shariffdeen, S. Rasnayaka, and N. Kasthuriarachchi, "Self-supervised vision transformers for malware detection," *IEEE Access*, vol. 10, pp. 103 121–103 135, 2022.

[11] I.-W. Jeong, H.-J. Lee, G.-N. Kim, and S.-H. Choi, "MalFormer: A novel vision transformer model for robust malware analysis," *IEEE Access*, vol. 13, pp. 122 671–122 683, 2025.

[12] H. Aghakhani *et al.*, "When malware is packin' heat; limits of machine learning classifiers based on static analysis features," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2020.

[13] S. Han, H. Yun, and Y. Park, "Deep learning for cybersecurity classification: Utilizing depth-wise CNN and attention mechanism on VM-obfuscated data," *Electronics*, vol. 13, no. 17, p. 3393, 2024.

[14] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 1597–1607.

[15] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," arXiv:1804.04637, 2018. [Online]. Available: https://arxiv.org/abs/1804.04637

[16] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas, "Malware detection by eating a whole EXE," in *Workshops of the 32nd AAAI Conf. Artificial Intelligence (AI for Cyber Security Workshop)*, New Orleans, LA, USA, 2018, arXiv:1710.09435. [Online]. Available: https://arxiv.org/abs/1710.09435

[17] J. Saxe and K. Berlin, "Deep neural network based malware detection using two dimensional binary program features," in *Proc. 10th Int. Conf. Malicious Unwanted Softw. (MALWARE)*, 2015, pp. 11–20.

[18] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[19] E. Raff, J. Sylvester, and C. Nicholas, "Learning the PE header, malware detection with minimal domain knowledge," in *Proc. 10th ACM Workshop Artif. Intell. Secur. (AISec)*, Dallas, TX, USA, 2017, pp. 121–132.

[20] G. D. Webster *et al.*, "Finding the needle: A study of the PE32 rich header and respective malware triage," in *Proc. Int. Conf. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, ser. Lecture Notes in Computer Science, vol. 10327, 2017, pp. 119–138.

[21] M. Sharif, A. Lanzi, J. Giffin, and W. Lee, "Automatic reverse engineering of malware emulators," in *Proc. 30th IEEE Symp. Security and Privacy (S&P)*, 2009, pp. 94–109.

[22] D. Xu, J. Ming, Y. Fu, and D. Wu, "VMHunt: A verifiable approach to partially-virtualized binary code simplification," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 442–458.

[23] D. Gibert, N. Totosis, C. Patsakis, Q. Le, and G. Zizzo, "Assessing the impact of packing on static machine learning-based malware detection and classification systems," *Computers & Security*, vol. 156, p. 104495, 2025.

[24] Y. Wu, S. Dou, D. Zou, W. Yang, W. Qiang, and H. Jin, "Contrastive learning for robust Android malware familial classification," *IEEE Transactions on Dependable and Secure Computing*, 2022, early access, Nov. 4, 2022.

[25] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[26] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, USA, 2019, pp. 729–746.

[27] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[28] G. Laurenza, R. Lazzeretti, and L. Mazzotti, "Malware triage for early identification of advanced persistent threat activities," *Digital Threats: Research and Practice*, vol. 1, no. 3, pp. 1–17, 2020.

[29] H. Haddadpajouh, A. Azmoodeh, A. Dehghantanha, and R. M. Parizi, "MVFCC: A multi-view fuzzy consensus clustering model for malware threat attribution," *IEEE Access*, vol. 8, pp. 139 188–139 198, 2020.

[30] L. Demetrio, B. Biggio, G. Lagorio, F. Roli, and A. Armando, "Functionality-preserving black-box optimization of adversarial windows malware," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3469–3478, 2021.

[31] L. Demetrio, S. E. Coull, B. Biggio, G. Lagorio, A. Armando, and F. Roli, "Adversarial EXEmples: A survey and experimental evaluation of practical attacks on machine learning for Windows malware detection," *ACM Trans. Privacy Secur.*, vol. 24, no. 4, pp. 1–31, 2021.

[32] D. Li, S. Cui, Y. Li, J. Xu, F. Xiao, and S. Xu, "PAD: Towards principled adversarial malware detection against evasion attacks," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 920–936, 2024.

[33] Y. Zhou, G. Cheng, Z. Chen, and S. Yu, "MalPurifier: Enhancing Android malware detection with adversarial purification against evasion attacks," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 4, pp. 9241–9258, 2026.

[34] A. Rashid and J. Such, "Effectiveness of moving target defenses for adversarial attacks in ML-based malware detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 4, pp. 3885–3900, 2025.

[35] A. Abusnaina *et al.*, "DL-FHMC: Deep learning-based fine-grained hierarchical learning approach for robust malware classification," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3432–3447, 2022.

[36] H. S. Anderson, A. Kharkar, B. Filar, D. Evans, and P. Roth, "Learning to evade static PE machine learning malware models via reinforcement learning," arXiv:1801.08917, 2018. [Online]. Available: https://arxiv.org/abs/1801.08917

[37] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[39] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[40] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 6105–6114.

[41] Z. Liu *et al.*, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 9992–10 002.

[42] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 10 347–10 357.

[43] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 991–25 004.

[44] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30. Curran Associates, Inc., 2017.

[45] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," arXiv:1807.03748, 2018. [Online]. Available: https://arxiv.org/abs/1807.03748
[46] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.
[47] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101
[48] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 507–520.



Xubo Fang received the B.Eng. degree in automation from the School of Electrical and Information Engineering, Zhengzhou University, Zhengzhou, China, in 2022. He is currently pursuing the M.E. degree in electrical, information, and communication engineering with the Graduate School of Natural Science and Technology, Kanazawa University, Kanazawa, Japan. His research interests include machine learning, deep learning, time-series forecasting, and uncertainty quantification. His GitHub page is available at https://github.com/boukuhaku.



Haochen Dong received the B.S. degree from the School of Aeronautical and Mechanical Engineering, Changzhou Institute of Technology, Changzhou, China, in 2023, and the M.S. degree from the College of Mechanical and Electrical Engineering, Hohai University, Nanjing, China, in 2026. He is currently pursuing the Ph.D. degree with the School of Oceanography, Shanghai Jiao Tong University, Shanghai, China. His research interests include robot learning and vision-based perception for autonomous robots. He is a Student Member of the IEEE.



Lei Zhong received the B.Eng. degree from the School of Mechanical and Electrical Engineering, Xuzhou Institute of Technology, Xuzhou, China, in 2018, and the M.E. degree from the College of Mechanical and Electrical Engineering, Hohai University, Changzhou, China, in 2021. He is currently with Hohai University, China.



Jiaqing Wang received the B.Eng. degree in mechanical engineering from the School of Mechanical Engineering and Automation, Northeastern University, Shenyang, China, in 2021, and the M.E. degree from Hohai University, Changzhou, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Oceanography, Shanghai Jiao Tong University, Shanghai, China. His research interests include robot learning, robotic perception, and robot simulators. His GitHub page is available at https://github.com/panzerjagerWang.



Sicheng Han received the B.Eng. degree in automation from Shandong University, Weihai Campus, Weihai, China, in 2017, and the M.S. degree in computer software from Hanyang University, Seoul, Republic of Korea, in 2024. He has practical experience in security operations governance, international education program operation, and cross-border business cooperation. His research interests include software security, deep learning for cybersecurity, and digital forensics.